PID CONTROLLER
DESIGN AND IMPLEMENTATION

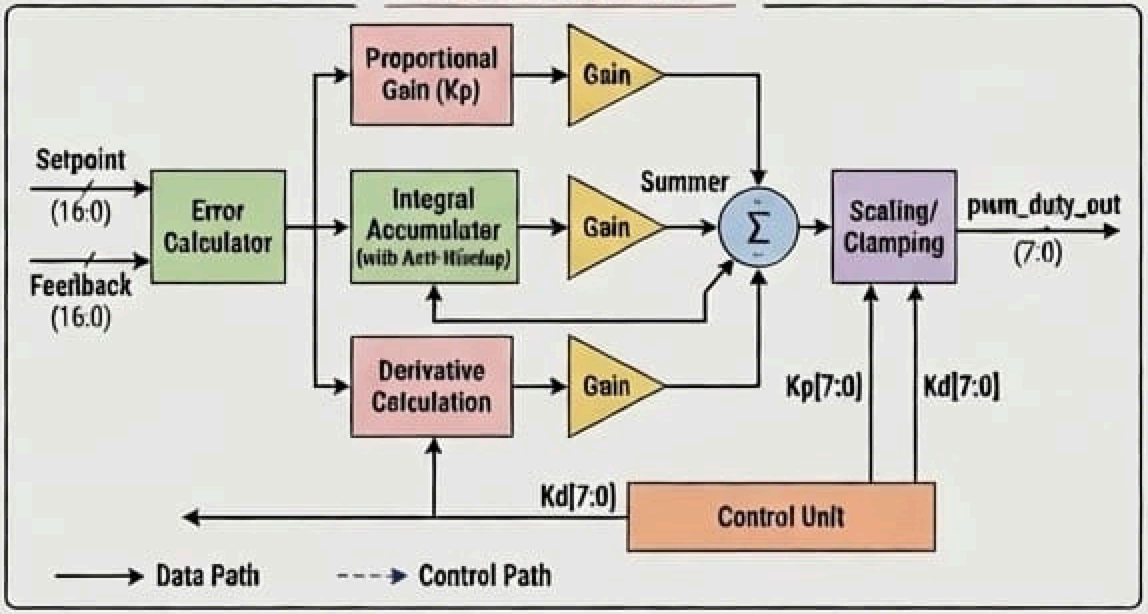
INTRODUCTION

PID controllers is an pen-instructen architecture (ISA) low sopibility, scalability and stalability, demond for battertly desiign, pow consonnous derices, luw the most important ent ments in digital systems.

OBJECTIVES

- To design a hardware-efficient PID controller.
- To implement anti-windop clamping and error.
- To simulate and verify with a realistic scenario.
- To compute assic basic la; boad precessor using buhanti-meter scenario.

BLOCK DIAGRAM



Setpoint (16:0)
Feelback (16:0)
Error Calculator
Proportional Gain (Kp)
Integral Accumulater (with Anti-Windup)
Derivative Calculation
Gain
Gain
Gain
Summer
Σ
Scaling/ Clamping
pwm_duty_out (7:0)
Kp[7:0]
Kd[7:0]
Kd[7:0]
Control Unit
⟶ Data Path
- - -› Control Path

WORKING FLOW

① Fetch Setpoint & Feedback in Setpoint
② Calculate Error in error error
③ Compute P, I, D terms in KI[7:0]
④ Sum and Clamping to cress setpoint
⑤ Sum and Clamping to sun set clamp
⑥ Output PWM to arress selpoint
⑧ Update integral in update integral

FEATURES

- ✓ Suppert signed 16-bit inputs
- ✓ Fixed-point architecture
- ✓ Anti-windup clamping
- ✓ Scaled gain parameters
- ✓ PWM duty cycle output

APPLICATIONS

DC Motor Control

Temperature Control

Robotic joint actuation

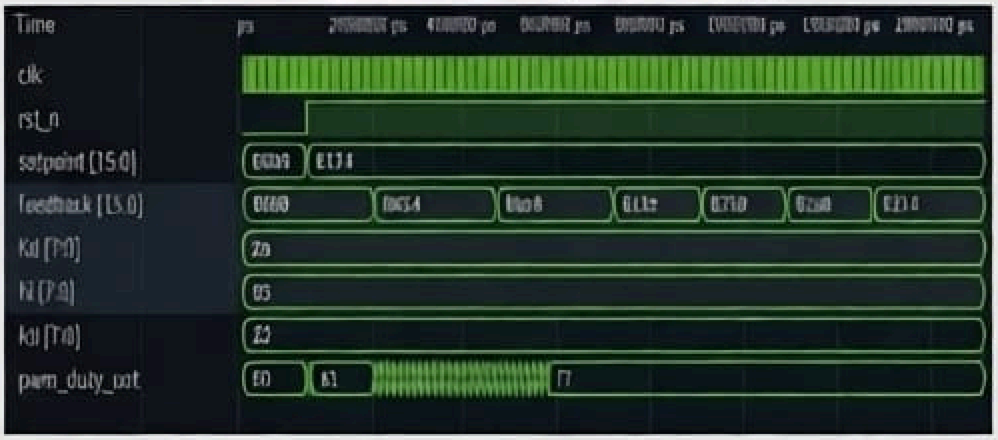
PID CONTROLLER (PID) CODE

```
module pid_controller (
   input clk,                  // Clock signal
   input rst_n,                // Active low
reset
   // Setpoints and Feedback
   input signed [15:0] setpoint,
 // Desired target value
   input signed [15:0] feedback,
 // Measured value from sensor
   // PID Parameters (Tuning Constants)
   input [7:0] Kp,
 // Proportional gain (scaled up by 2^8)
   input [7:0] Ki,
 // Integral gain (scaled up by 2^8)
   input [7:0] Kd,
 // Derivative gain (scaled up by 2^8)
 // Control Outputs
   output [7:0] pwm_duty_out
// Final PWM output (0-255)
);
   localparam SCALING_FACTOR = 8;
   // State registers
   reg signed [16:0] error_prev;
   reg signed [31:0] error_integral;
 // Combinational wires for immediate
calculation
   wire signed [16:0] error_curr;
   wire signed [16:0] error_derivative;
   wire signed [24:0] p_term;
   wire signed [39:0] i_term;
   wire signed [24:0] d_term;
   wire signed [41:0] pid_out_raw;
   wire signed [41:0] pid_scaled;
 // 1. Instantaneous Error Calculations
 assign error_curr = setpoint - feedback;
 assign error_derivative = error_curr -
error_prev;
 // 2. Structural Sign Extensions for
Multiplications
 assign p_term = error_curr * $signed({1'b0,
Kp});
 assign d_term = error_derivative *
$signed({1'b0, Kd});
 assign i_term = error_integral *
$signed({1'b0, Ki});
 // 3. Sum the Terms
 assign pid_out_raw = p_term + i_term +
d_term;
 // Arithmetic right shift to handle fixed-
point scaling
 assign pid_scaled = pid_out_raw >>>
SCALING_FACTOR;
 // 4. Synchronous State Updates (Single
Block)
 always @(posedge clk or negedge rst_n)
begin // <-- FIXED 'negedge' TYPO HERE
 if (~rst_n) begin
 error_prev <= 0;
 error_integral <= 0;
 end else begin
 error_prev <= error_curr;
 // Anti-windup clamping to protect integral
accumulator
 if (error_curr > -500 && error_curr < 500)
begin
 if (error_integral > 32'sh000FFFFF)
 error_integral <= 32'sh000FFFFF;
 else if (error_integral < -32'sh000FFFFF)
 error_integral <= -32'sh000FFFFF;
 else
 error_integral <= error_integral + error_curr;
 end   end   end
 // 5. Safe Combinational Output Clamping
(0 to 255)
 assign pwm_duty_out = (pid_scaled[41]) ?
8'd0 :
 (pid_scaled > 255) ? 8'd255 :
 pid_scaled[7:0];
endmodule
```

TESTBENCH CODE

```
`timescale 1ns / 1ps
module pid_controller_tb;
   // Testbench signals
   reg clk;
   reg rst_n;
   reg signed [15:0] setpoint;
   reg signed [15:0] feedback;
   reg [7:0] Kp;
   reg [7:0] Ki;
   reg [7:0] Kd;
   wire [7:0] pwm_duty_out;
   // Instantiate Unit Under Test (UUT)
   pid_controller uut (
      .clk(clk),
      .rst_n(rst_n),
      .setpoint(setpoint),
      .feedback(feedback),
      .Kp(Kp),
      .Ki(Ki),
      .Kd(Kd),
      .pwm_duty_out(pwm_duty_out)
   );
   // Clock generation (100MHz - 10ns
period)
   always #5 clk = ~clk;
   initial begin
      // Setup waveform dumping for
ModelSim/Questa
      $dumpfile("pid_simulation.vcd");
      $dumpvars(0, pid_controller_tb);
      // Initialize Inputs
      clk = 0;
      rst_n = 0;
      setpoint = 0;
      feedback = 0;
      // Tuning constants
      Kp = 8'd50;
      Ki = 8'd5;
      Kd = 8'd10;
      // Hold reset condition
      #100;
      rst_n = 1;
 // Monitor prints output in the ModelSim Transcript
window
 $monitor("Time=%0dns | Setpoint=%d | Feedback=%d |
PWM Out=%d", $time, setpoint, feedback,
pwm_duty_out);

 // --- TEST SCENARIO: Ramping up to target ---
 setpoint = 16'd500;

 #200 feedback = 16'd100;
 #200 feedback = 16'd200;
 #200 feedback = 16'd300;
 #200 feedback = 16'd400;
 #200 feedback = 16'd480;
 #200 feedback = 16'd500;
 #200 feedback = 16'd500;
 #200 feedback = 16'd505; // Minor overshoot simulation
 #200 feedback = 16'd500;
 #200;

 $display("PID Simulation Completed.");
 $finish;
 end

endmodule
```

SIMULATION RESULT (WAVEFORM)



CONCLUSION

The PID controller (PID) [illegible] has been successful design and verificatnd st and sohe PIIP clear processor arshlleation and verificded ts imponronche aod-power and bancis execution. This module and tirees plots and ent verify the processory and power heat levent rescolption.

SIMULATION SUMMARY

Time (ou)	Reset	Duth	Drarck	PV Tolue	Descriplion
0	1	1	0	0	Roost fuclee
25	0	1	0	4	Fout lectcastion
25	0	1	7	100	Soeast Taon
15	0	8	0	170	Rasth Eyrte